

Report for Hospital Management Database Design

Student Name: Corbu Ciprian - Constantin

Student Number: S24009126

Module Title: Database Systems

Module Code: CONL705

Abstract

This report provides an overview of the design decisions made in the hospital management database developed in SQL. It explains the rationale behind the selection of data types, keys, and constraints, discusses trade-offs encountered, and reflects on the challenges during the design process. Additionally, it examines the impact of these decisions on query performance. The objective was to develop a normalised, maintainable, and efficient relational database capable of handling core hospital operations including patient management, staff records, room allocation, appointments, admissions, treatments, and billing.

I. Introduction

Hospitals require reliable data systems to ensure patient safety, operational efficiency, and compliance. The database schema developed addresses these needs by enforcing data integrity, supporting scalability, and delivering efficient query performance. This report critically evaluates the core design choices and their implications.

II. Data Types Justification

A. Identifiers

Primary keys in all major tables (Patient, Doctor, Nurse, Room, Appointment, Admission, Treatment, Bill) use SERIAL. This ensures unique auto-incrementing integers, simplifying inserts and optimising indexing and JOIN operations[1].

B. Strings and Text Fields

- **Names and Contacts:** Fields like first_name, last_name, email, and phone_number use VARCHAR, appropriately sized to balance flexibility and storage efficiency.
- **Address:** VARCHAR(255) accommodates varied international address formats.
- **Descriptions:** TEXT allows for variable-length narrative content in fields like reason_for_visit and treatment_description.

C. Dates and Times

Temporal data is modelled precisely:

- DATE is used for birth dates, admissions, discharges, and treatments.
- TIME is used for appointment_time, ensuring precise scheduling.

D. Numerical Data

Financial fields (room_charge, bill_amount) use DECIMAL(10,2) to guarantee accuracy in billing calculations. Foreign keys use INT, promoting efficient indexing and referential integrity.

E. Status Fields

Fields such as gender, availability_status, and payment_status use VARCHAR with CHECK constraints. This balances flexibility with data integrity, allowing for future adaptations[2].

III. Keys and Constraints

A. Primary Keys

Primary keys are uniformly applied to *_id fields, supporting efficient data access and enforcing uniqueness.

B. Foreign Keys

Foreign keys maintain referential integrity across key relationships:

- Appointment links to Patient and Doctor.
- Admission links to Patient, Room, and Doctor.
- Treatment links to Admission.
- Bill links to Admission.

C. Unique Constraints

- phone_number and email fields are unique in Patient, Doctor, and Nurse tables.
- The Appointment table enforces a composite UNIQUE constraint on (appointment_date, appointment_time, doctor_id) to prevent double-booking.

D. CHECK Constraints

CHECK constraints enforce domain rules, such as valid statuses and non-negative charges. These strengthen integrity at the database level.

IV. Trade-offs Considered

A. VARCHAR + CHECK vs ENUM

VARCHAR with CHECK constraints was chosen over ENUMs for greater schema flexibility and easier future updates.

B. TEXT vs VARCHAR

TEXT is used for narrative fields, while VARCHAR is reserved for fields with predictable lengths, improving indexing[3].

C. SERIAL vs UUID

SERIAL was preferred for simplicity and performance, as global uniqueness was not required.

D. Normalisation vs Denormalisation

The schema adheres to 3NF to reduce redundancy and preserve consistency, despite the potential need for additional JOINS in reporting.

V. Challenges Encountered

A. Managing Room Availability

Synchronising room availability required a trigger on Admission to update availability_status automatically, ensuring real-time accuracy.

B. Appointment Scheduling

A composite UNIQUE constraint was necessary to prevent double-bookings. Handling time zones also required careful configuration.

C. Evolving Requirements

Using VARCHAR with CHECK constraints allows the schema to adapt to evolving healthcare needs, such as adding new statuses.

VI. Impact on Query Performance

A. Indexing

Indexes were added to improve query performance:

- `idx_patient_name` for name-based searches.
- `idx_doctor_specialisation` for filtering by specialty.
- `idx_appointment_date_time` for efficient appointment queries.

B. Views

Views such as `doctor_appointments` and `patient_bills` encapsulate common JOINS, facilitating reporting and ensuring query consistency.

C. Triggers and Procedures

Triggers (e.g., updating room availability) and stored procedures (e.g., `book_appointment`) ensure business logic is enforced consistently and efficiently.

D. Query Efficiency

Despite the schema's normalisation, performance remains strong due to well-planned indexing and optimised query patterns.

VII. Conclusion

The hospital database schema reflects a balanced approach to flexibility, integrity, and performance. CRUD operations were used to process the data. Each design decision supports robust, scalable hospital operations and accommodates future needs. The result is an efficient, maintainable system suitable for high-demand healthcare environments.

References:

- [1] A. Debarros, *Practical SQL: A Beginner's Guide to Storytelling with Data*, 2nd ed., San Francisco, CA, USA: No Starch Press, 2022.
- [2] PostgreSQL, "Database Initialization — initdb," PostgreSQL Documentation, 2024. [Online]. Available: <https://www.postgresql.org/docs/current/app-initdb.html>
- [3] P. J. Sadalage and M. Fowler, *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*, Addison-Wesley Professional, 1st ed., 2009.